

MSSQL Stored Functions Assignment: Student Database

Scenario

A university wants to use **Stored Functions** to calculate student GPA status, scholarship eligibility, exam results, and academic reports.

Question 1 — Grade Point Function

Create a function that accepts marks and returns grade point.

```
32 CREATE FUNCTION fn_GradePoint (@marks INT)
33 RETURNS DECIMAL(3,2)
34 AS
35 BEGIN
36     DECLARE @gradePoint DECIMAL(3,2);
37     IF @marks >= 80 SET @gradePoint = 4.0;
38     ELSE IF @marks >= 70 SET @gradePoint = 3.0;
39     ELSE IF @marks >= 60 SET @gradePoint = 2.0;
40     ELSE IF @marks >= 50 SET @gradePoint = 1.0;
41     ELSE SET @gradePoint = 0.0;
42     RETURN @gradePoint;
43 END;
44 GO
```

Question 2 — Grade Letter Function

Create a function that accepts marks and returns grade letter.

```
49 CREATE FUNCTION fn_GradeLetter (@marks INT)
50 RETURNS VARCHAR(2)
51 AS
52 BEGIN
53     DECLARE @gradeLetter VARCHAR(2);
54     IF @marks >= 80 SET @gradeLetter = 'A';
55     ELSE IF @marks >= 70 SET @gradeLetter = 'B';
56     ELSE IF @marks >= 60 SET @gradeLetter = 'C';
57     ELSE IF @marks >= 50 SET @gradeLetter = 'D';
58     ELSE SET @gradeLetter = 'F';
59     RETURN @gradeLetter;
60 END;
61 GO
```

Question 3 — Pass or Fail Function

Create a function that checks whether a student passed or failed.

```

66 CREATE FUNCTION fn_PassFail (@marks INT)
67 RETURNS VARCHAR(10)
68 AS
69 BEGIN
70     RETURN (CASE WHEN @marks >= 50 THEN 'Pass' ELSE 'Fail' END);
71 END;
72 GO

```

Question 4 — GPA Status Function

Create a function that accepts GPA and returns academic status.

```

77 CREATE FUNCTION fn_GPASStatus (@gpa DECIMAL(3,2))
78 RETURNS VARCHAR(20)
79 AS
80 BEGIN
81     DECLARE @status VARCHAR(20);
82     IF @gpa >= 3.5 SET @status = 'Excellent';
83     ELSE IF @gpa >= 3.0 SET @status = 'Good';
84     ELSE IF @gpa >= 2.0 SET @status = 'Average';
85     ELSE SET @status = 'Poor';
86     RETURN @status;
87 END;
88 GO

```

Question 5 — Scholarship Eligibility Function

Create a function that checks whether a student is eligible for scholarship.

```

93 CREATE FUNCTION fn_ScholarshipEligibility (@gpa DECIMAL(3,2))
94 RETURNS BIT
95 AS
96 BEGIN
97     RETURN (CASE WHEN @gpa >= 3.5 THEN 1 ELSE 0 END);
98 END;
99 GO

```

Question 6 — Age Calculation Function

Create a function that calculates student age from date of birth.

```

104 CREATE FUNCTION fn_CalculateAge (@dob DATE)
105 RETURNS INT
106 AS
107 BEGIN
108     RETURN DATEDIFF(YEAR, @dob, GETDATE());
109 END;
110 GO

```

Question 7 — Full Name Function

Create a function that combines first name and last name.

```

115 CREATE FUNCTION fn_FullName (@firstName VARCHAR(50), @lastName VARCHAR(50))
116 RETURNS VARCHAR(100)
117 AS
118 BEGIN
119     RETURN @firstName + ' ' + @lastName;
120 END;
121 GO

```

Question 8 — Tuition Discount Function

Create a function that calculates discount amount based on GPA.

```

126 CREATE FUNCTION fn_TuitionDiscount (@gpa DECIMAL(3,2), @tuition MONEY)
127 RETURNS MONEY
128 AS
129 BEGIN
130     DECLARE @discount MONEY;
131     IF @gpa >= 3.8 SET @discount = @tuition * 0.20; -- 20% discount
132     ELSE IF @gpa >= 3.5 SET @discount = @tuition * 0.10; -- 10% discount
133     ELSE SET @discount = 0;
134     RETURN @discount;
135 END;
136 GO

```

```

138 -----
139 -- Test Queries
140 -----
141 SELECT
142     student_id,
143     dbo.fn_FullName(first_name, last_name) AS FullName,
144     dbo.fn_CalculateAge(date_of_birth) AS Age,
145     marks,
146     dbo.fn_GradePoint(marks) AS GradePoint,
147     dbo.fn_GradeLetter(marks) AS GradeLetter,
148     dbo.fn_PassFail(marks) AS PassFail,
149     gpa,
150     dbo.fn_GPASStatus(gpa) AS GPASStatus,
151     dbo.fn_ScholarshipEligibility(gpa) AS ScholarshipEligible,
152     tuition_fee,
153     dbo.fn_TuitionDiscount(gpa, tuition_fee) AS DiscountAmount
154 FROM students;
155 GO

```

100 % No issues found

Results

Messages

	student_id	FullName	Age	marks	GradePoint	GradeLetter	PassFail	gpa	GPASStatus	ScholarshipEligible	tuition_fee	DiscountAmount
1	1	Aung Aung	21	85	4.00	A	Pass	3.80	Excellent	1	500000.00	100000.00
2	2	Su Su	20	72	3.00	B	Pass	3.20	Good	0	500000.00	0.00
3	3	Kyaw Kyaw	22	45	0.00	F	Fail	2.10	Average	0	500000.00	0.00
4	4	Hla Hla	21	90	4.00	A	Pass	3.95	Excellent	1	500000.00	100000.00

Database Setup

```
CREATE DATABASE StudentFunctionDB;
```

```
GO
```

```
USE StudentFunctionDB;
```

```
GO
```

```
CREATE TABLE students (  
    student_id INT PRIMARY KEY IDENTITY(1,1),  
    first_name VARCHAR(50),  
    last_name VARCHAR(50),  
    date_of_birth DATE,  
    marks INT,  
    gpa DECIMAL(3,2),  
    tuition_fee MONEY  
);  
GO
```

```
INSERT INTO students  
(first_name, last_name, date_of_birth, marks, gpa, tuition_fee)  
VALUES  
( 'Aung', 'Aung', '2005-01-15', 85, 3.80, 500000),  
( 'Su', 'Su', '2006-03-20', 72, 3.20, 500000),  
( 'Kyaw', 'Kyaw', '2004-07-10', 45, 2.10, 500000),  
( 'Hla', 'Hla', '2005-11-05', 90, 3.95, 500000);  
GO
```
